

CS 307: Software Engineering

Homework #2

Due on March 5th, 2026 at 11:58pm

Team #26: BoilerBridge

Team Members: Logan, Shreyansh, Xavion Marable, Emilio Guzman, Shashank M

Problem 1

This is a group homework problem. Your group will be the same as the one that you are enrolled in for the team project in this course.

Find one open-source project that is closest to your team project. Reverse engineer the design and a few architectural/design patterns from that project.

Describe the above reverse engineered details in the following format. If applicable, provide pointers to project's directories/files/classes in your descriptions and/or diagrams.

1. Purpose (1 point)

a. Briefly explain the chosen open-source project and its purpose.

This is an AI powered travel planning application that is designed to help users organize and manage trips more easily. This is a full-stack, React Native application that uses generative AI to create personalized travel recommendations and assist users in planning itineraries. This application automatically suggests destinations, activities and accommodations based on user preferences using AI. Its purpose is to let users log in, create trips/itineraries, get AI-generated travel plans (Gemini) and use Google locations services while storing trip and user data in Firebase.

b. Provide the link to the open-source project repository.

<https://github.com/zahraaMeky/ai-travel-planner-app>

2. Design Outline (2 points)

a. Identify its components and describe the purpose of each component.

There are 8 main components for this React-Native application, with each one of them having subcomponents.

1. **app/** → Holds the main application screens and routing logic that controls navigation between features like authentication, trip creation, trip details and the main tab interface
2. **components/** → Holds reusable UI components and feature modules like login and trip creation that are used across multiple screen in the app.
3. **config/** → Stores the configuration files for external services

like Gemini and Firebase. Enables authentication, database access and AI generated travel recommendations

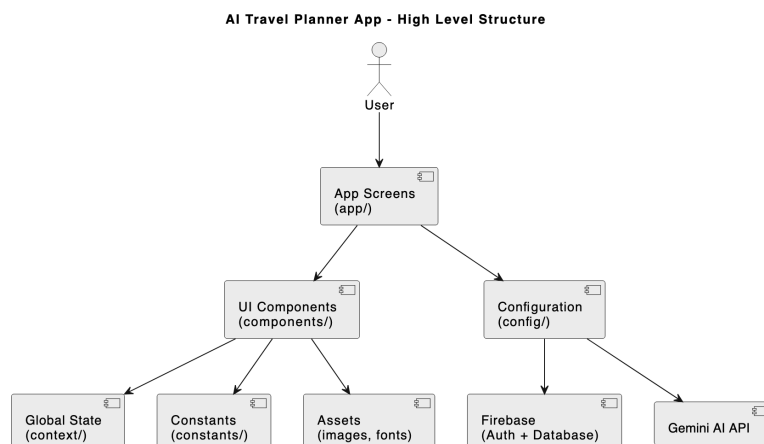
4. **context/** → Implements React Context to manage and share global state, like trip creation data, across different parts of the application.
5. **constants/** → Holds shared constants like colors and predefined data to ensure a consistent style and reusable values in the app.
6. **assets/** → Contains static resources like images and fonts that are used by the UI.
7. **scripts/** → Holds utility scripts used for development tasks like resetting or maintaining the project.
8. **Root Configuration Files** → Holds files like package.json, app.json that manage project dependencies, application settings and build configurations.

b. Describe the interactions between components.

The **app/** component is the central controller of the app. It loads the different screens and uses the **components/** folder to render the user interface elements for login, trip creating, viewing trip details, etc. When creating a trip or logging in, the screens call services that are defined in the **config/** directory. The configuration files connect the app to external services like Firebase for auth and database storage, and the Gemini AI for generating travel itineraries.

The **context/** module manages shared data, like trip information, and allows multiple screens and UI components to access and update that data without passing it manually between files. The **constants/** components provide shared values like colors and predefined data that are used by components and screens to maintain consistent design and behaviour. Additionally, the **assets/** components provide the images and fonts used by the UI. Finally the **scripts/** and **root configuration files** support the development and execution of the project by managing dependencies, build settings and utility tasks needed to run the application.

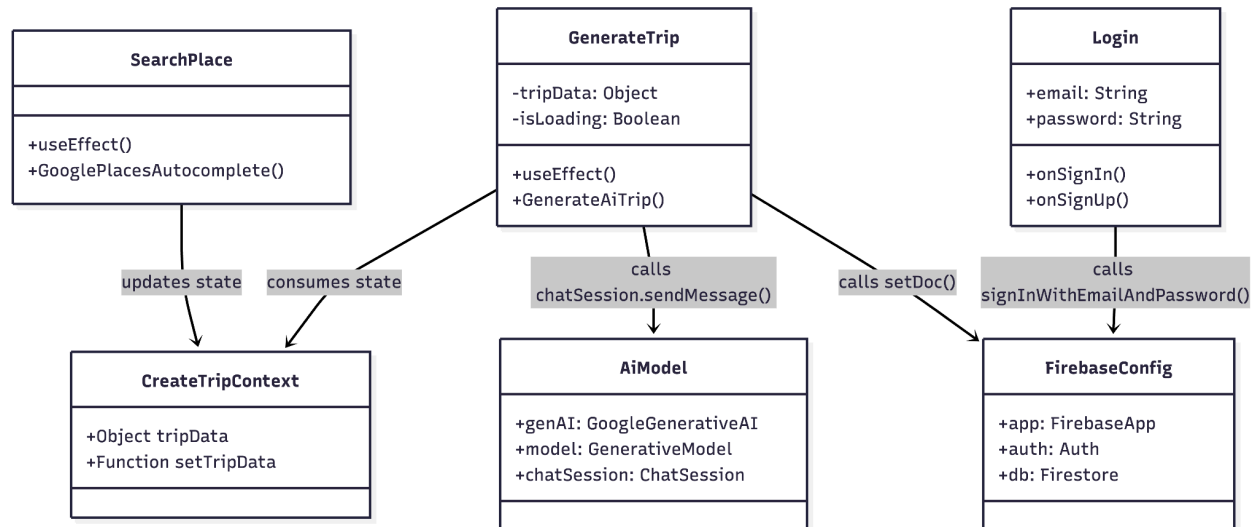
c. Include at least one UML diagram that clearly shows high-level structure of the system.



3. Design Details (7 points)

a. Include class level design (i.e., class diagrams) and be as detailed as you can.

Detailed Class Diagram

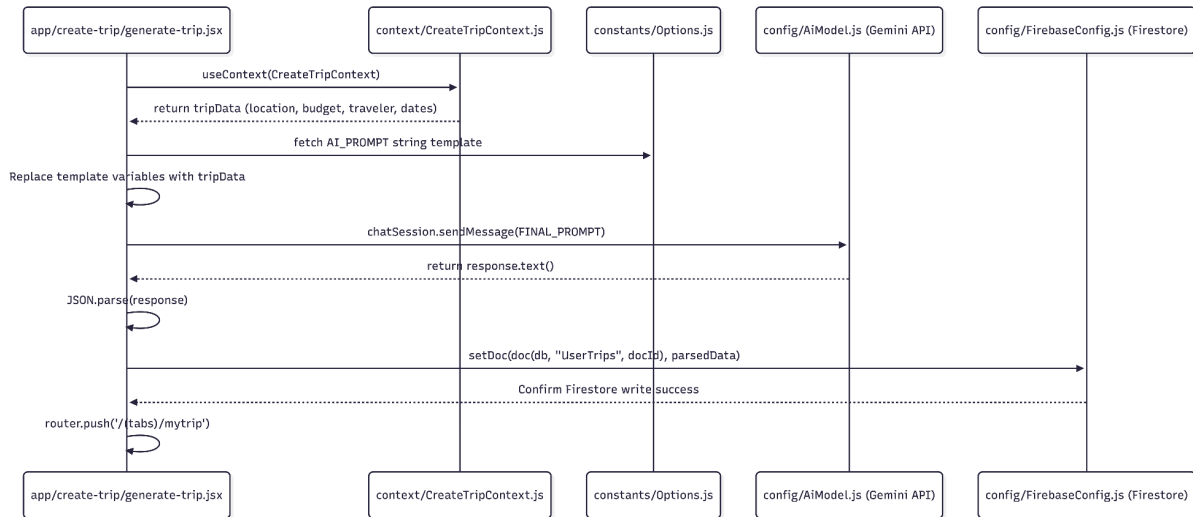


b. Describe the classes and interactions between the classes.

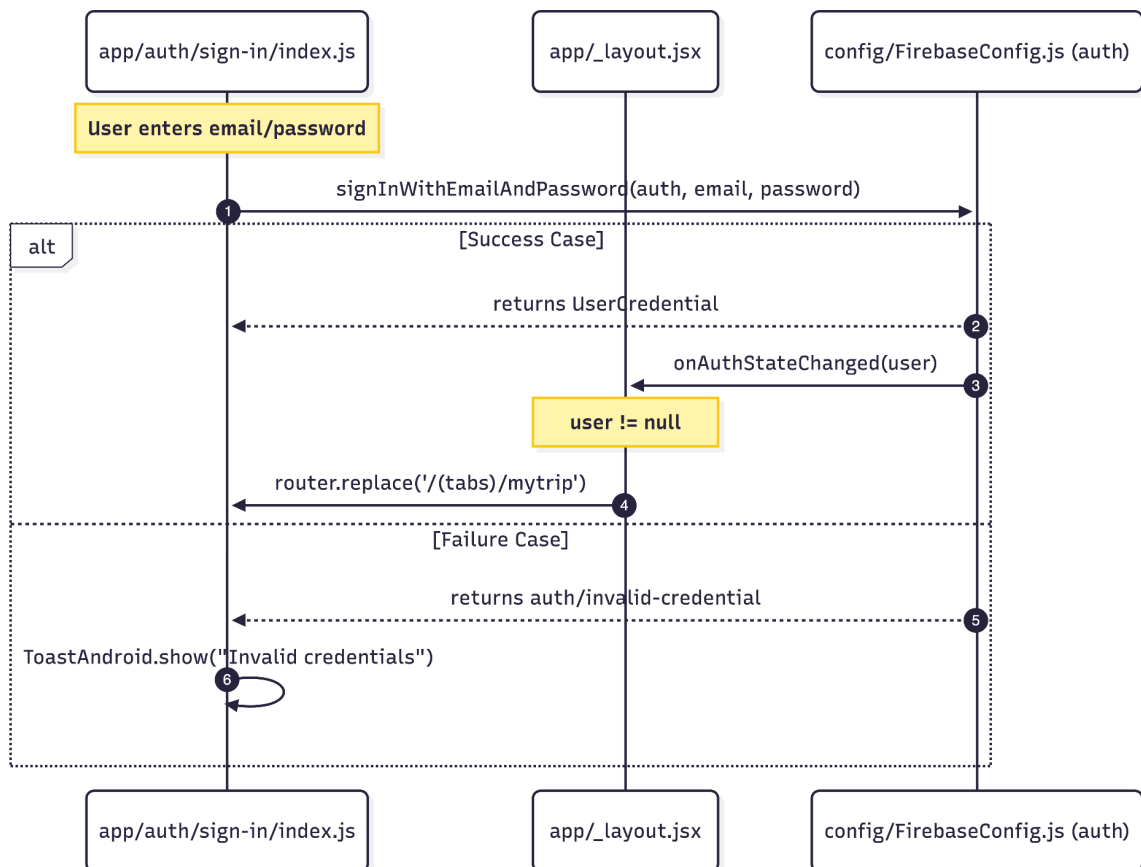
- **CreateTripContext**: This module in `context/CreateTripContext.js` acts as the main storage for a trip while a user is still picking their options. It holds the `userTripData` object and provides the `setUserTripData` function so every screen in the flow can update the destination, budget, or traveler count.
- **GenerateTrip**: This component in `app/create-trip/generate-trip.jsx` handles the logic of turning user choices into a final plan. It reads from the context, sends the formatted data to the AI, and manages the process of parsing the result and saving it to the database.
- **AiModel**: Located in `config/AiModel.js`, this file sets up the connection to the Gemini API. It configures the `AiChatSession` with the specific parameters and safety settings required to get a structured JSON response from the model.
- **FirebaseConfig**: This configuration file in `config/FirebaseConfig.js` initializes the Firebase app. It exports the `auth` and `db` instances used by the login screens and the trip generation logic to communicate with the backend.
- **SearchPlace & Login**: These are input-focused classes. **SearchPlace** uses the Google Places API to update location data in the context, while **Login** (in `app/auth/sign-in/index.js`) manages user credentials through the Firebase authentication service.

c. Include sequence diagrams for different activities and/or activity diagrams and/or state diagrams (at least five UML diagrams).

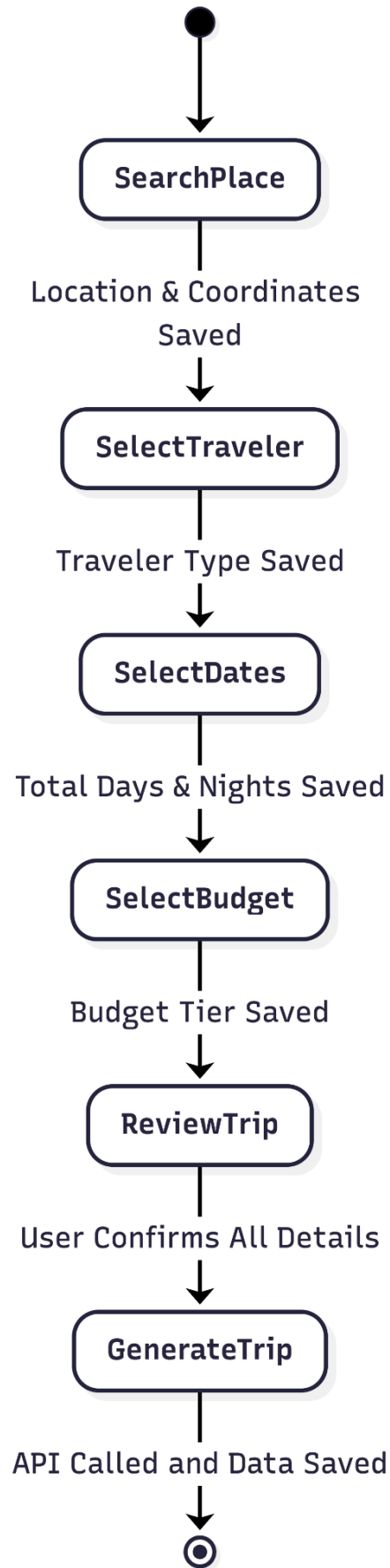
Sequence Diagram 1 - Trip Generation Flow



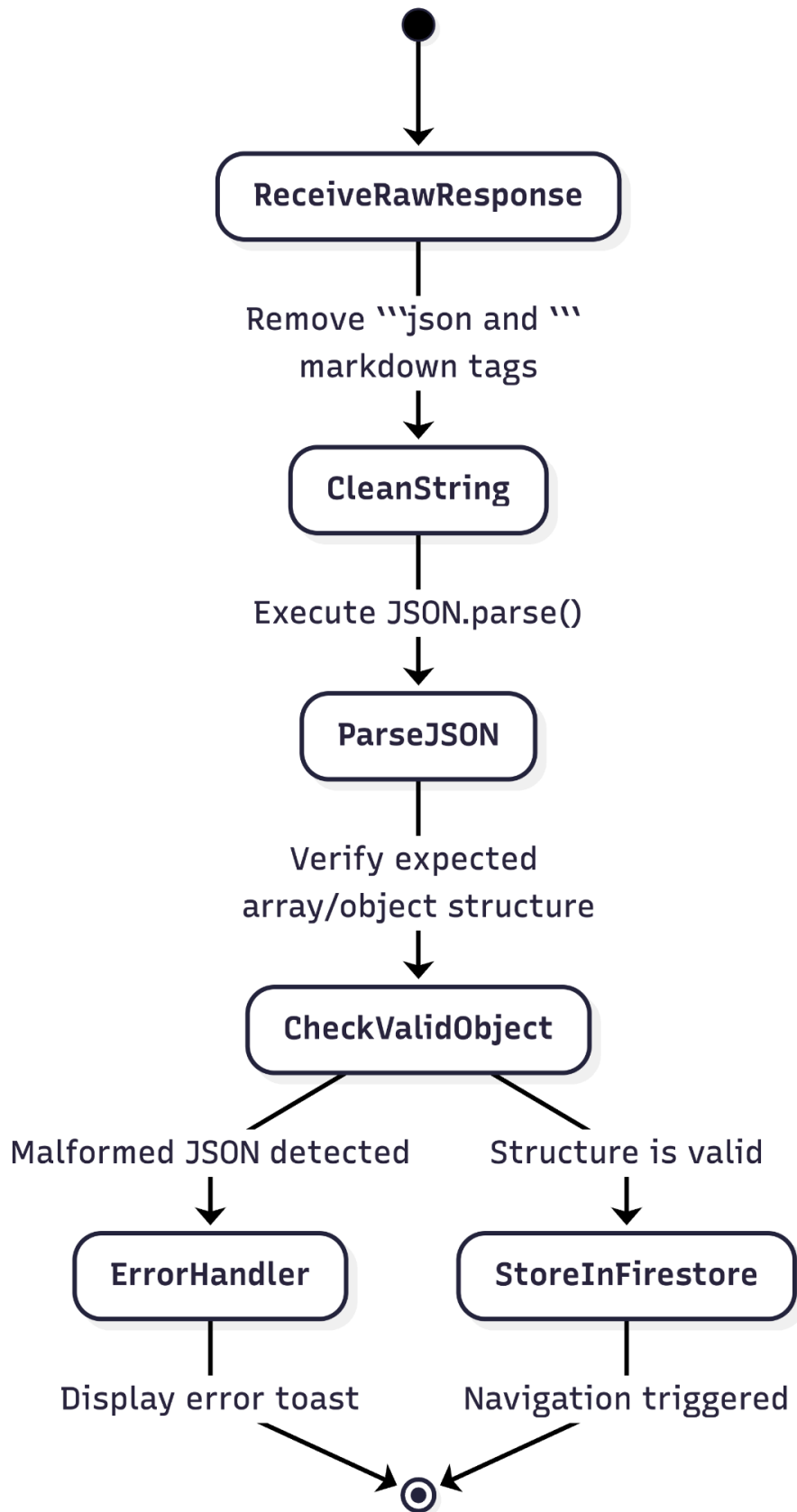
Sequence Diagram 2 - Authentication Flow



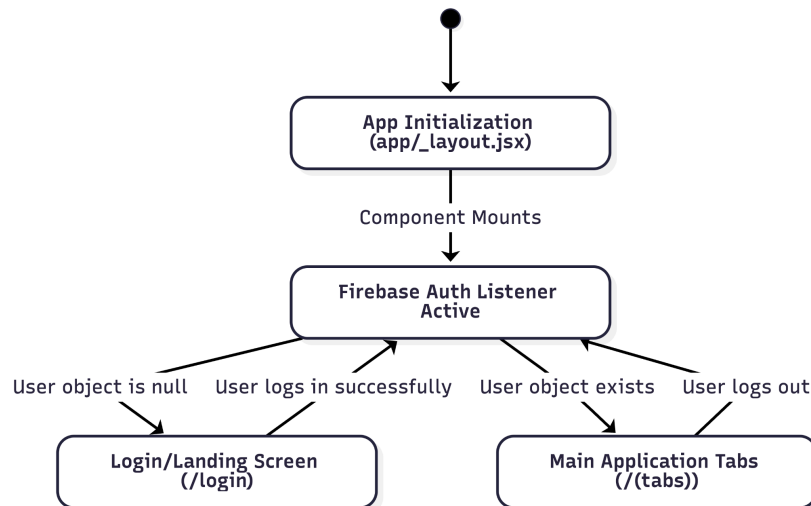
Sequence Diagram 3 - Multi-Step Form Flow



Sequence Diagram 4 - AI Response Parsing Logic



Sequence Diagram 5 - Application Authentication Status



d. In your descriptions and/or diagrams, point to the source file and line numbers if applicable, so that the grader can have a look.

1. AI Trip Generation (Diagram 1)

- Constructing the Prompt: In `app/create-trip/generate-trip.jsx` at Line 24, the component pulls the `AI_PROMPT` template from `constants/Options.js` and injects the user's trip preferences.
- Executing the API Call: The actual request to the Gemini API happens on Line 33 of `app/create-trip/generate-trip.jsx` through the `AiChatSession.sendMessage` method.
- Saving to the Database: Once the itinerary is received, it is persisted to the "UserTrips" collection in Firestore on Line 42 using the `setDoc()` method.

2. Authentication Logic (Diagram 2)

- The Sign-In Call: The interaction with Firebase Auth is located on Line 27 of `app/auth/sign-in/index.js`, where `signInWithEmailAndPassword` is invoked.
- Error Handling: The logic for invalid credentials (the else path in the sequence diagram) is handled by the catch block on Lines 22-43, which triggers a `ToastAndroid` notification.

3. Multi-Step Form State (Diagram 3)

- Updating Global State: Each input screen in the `app/create-trip/` directory (such as `select-budget.jsx`) calls `setUserTripData` from `context/CreateTripContext.js` to update the central store.
- Navigation Flow: The sequential movement between screens is managed by `router.push()` calls found at the conclusion of each selection function within those specific screen files.

4. Cleaning the AI Response (Diagram 4)

- Scrubbing Markdown: Gemini often returns the itinerary wrapped in markdown code blocks, so lines 24-30 of `app/create-trip/generate-trip.jsx` use `.replace()` to strip out the ````json` and ````` tags.
- Parsing the String: The sanitized string is converted into a standard JavaScript object on Line 36 using `JSON.parse()`.

5. Navigation Guard (Diagram 5)

- App Initialization (Init): The process begins in `app/_layout.jsx` where the app

sets up the global context on Line 16.

b. Login State: Because there is no automatic listener in `_layout.jsx`, the app defaults to the login route defined in the Stack on Line 19.

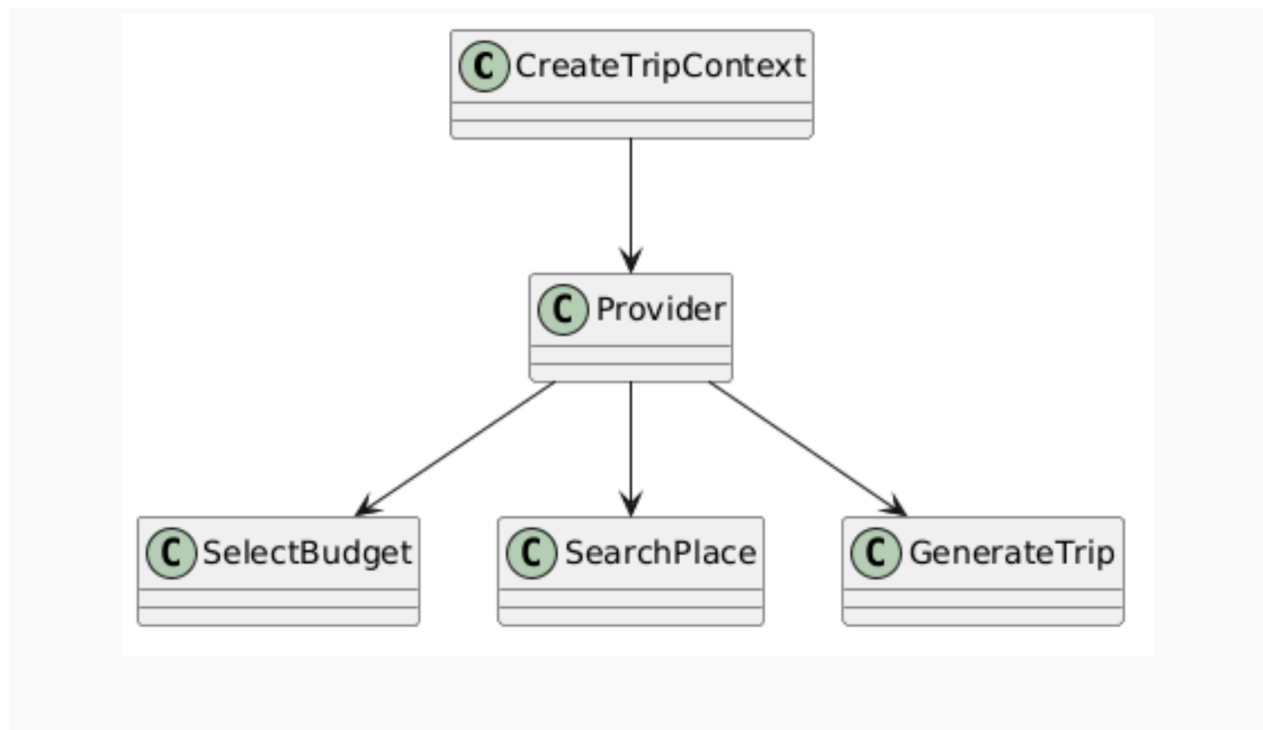
c. State Transition (Login to Main App): The move from the "Login" state to the "Main App" state is triggered in `app/auth/sign-in/index.js` on Line 47. When the sign-in succeeds, a manual `router.replace` call pushes the user into the (tabs) group.

d. Main Application State: This state is active when the user is within the (tabs) component, which is registered as a valid route on Line 20 of `app/_layout.jsx`.

4. Design Patterns (4 points)

a. Provide textual description of the patterns along with their diagrams (at least five patterns).

Pattern 1 — Context Pattern



The application uses the React Context Pattern to share trip creation data between multiple components without passing props through intermediate layers. The `CreateTripContext` object provides a centralized state that is accessed by different trip creation screens.

GitHub reference:

<https://github.com/zahraaMeky/ai-travel-planner-app/blob/main/context/CreateTripContext.jsx#L1-L2>

Pattern 2 — Singleton Pattern (Firebase Configuration)

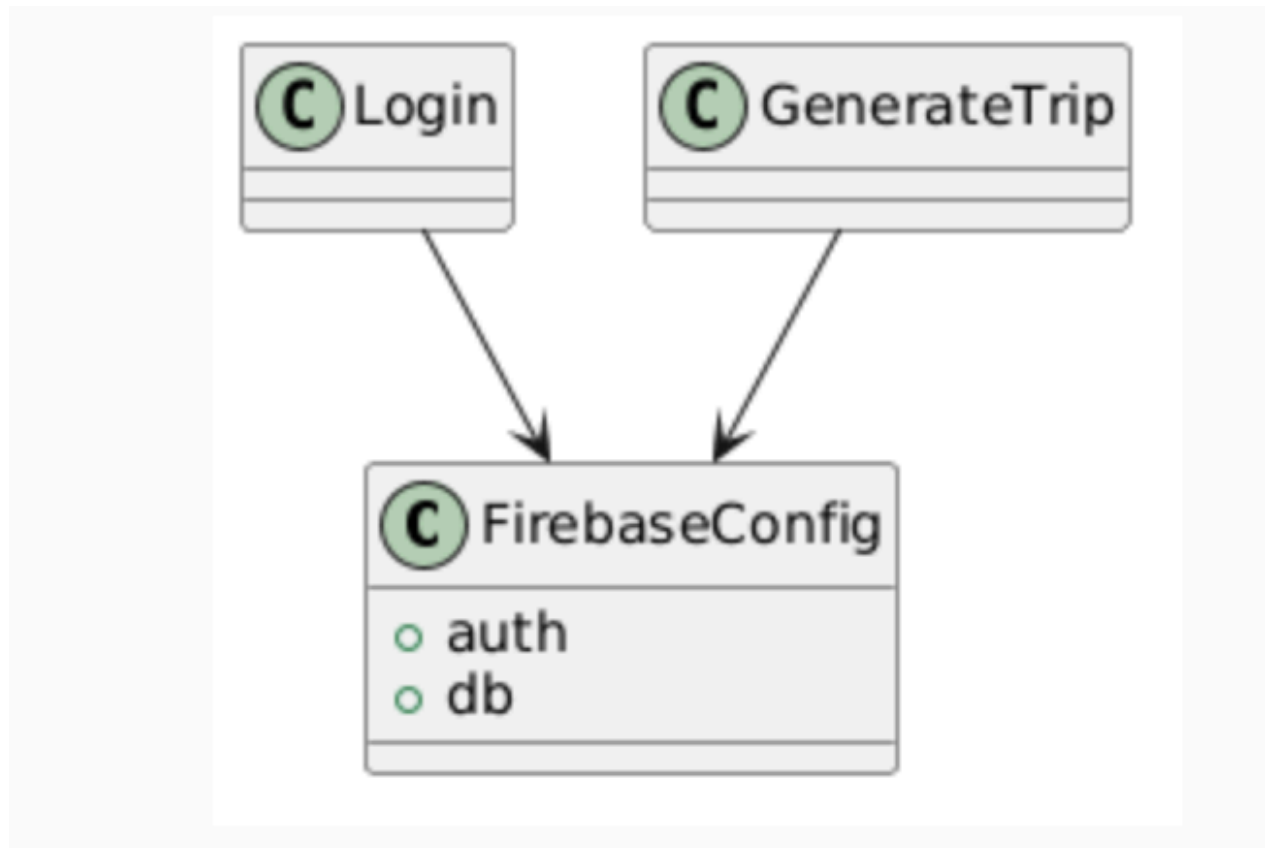
The Firebase configuration acts as a Singleton, ensuring that only one instance of the Firebase application is initialized throughout the project.

This prevents duplicate connections and ensures that all components share the same

authentication and database instance.

GitHub reference:

<https://github.com/zahraaMeky/ai-travel-planner-app/blob/d77de657ae0fc65de66386d52115b483528f8bdb/config/FirebaseConfig.js#L1>



Pattern 3 — Facade Pattern (AI Model Wrapper)

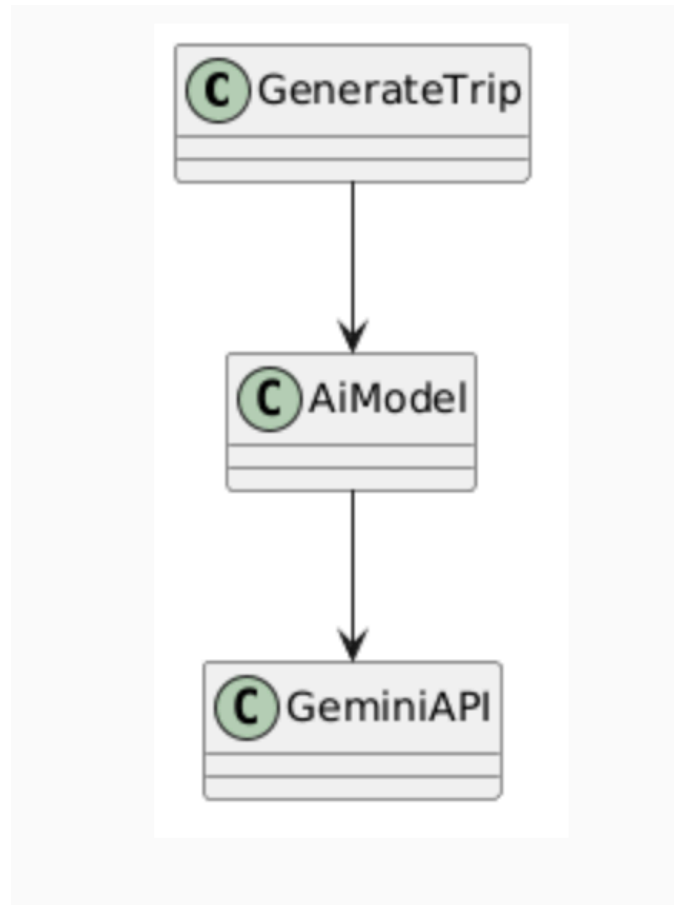
The **AiModel** module acts as a **Facade** that simplifies communication with the Gemini API. Instead of exposing complex API configuration details to UI components, the application provides a simplified interface for sending prompts and receiving travel plans.

This abstraction reduces coupling between UI components and external services.

GitHub reference:

<https://github.com/zahraaMeky/ai-travel-planner-app/blob/main/app/create-trip/Generate-Trip.jsx#L7>

<https://github.com/zahraaMeky/ai-travel-planner-app/blob/main/app/create-trip/Generate-Trip.jsx#L36>



Pattern 4 — Adapter Pattern (AI Response Processing)

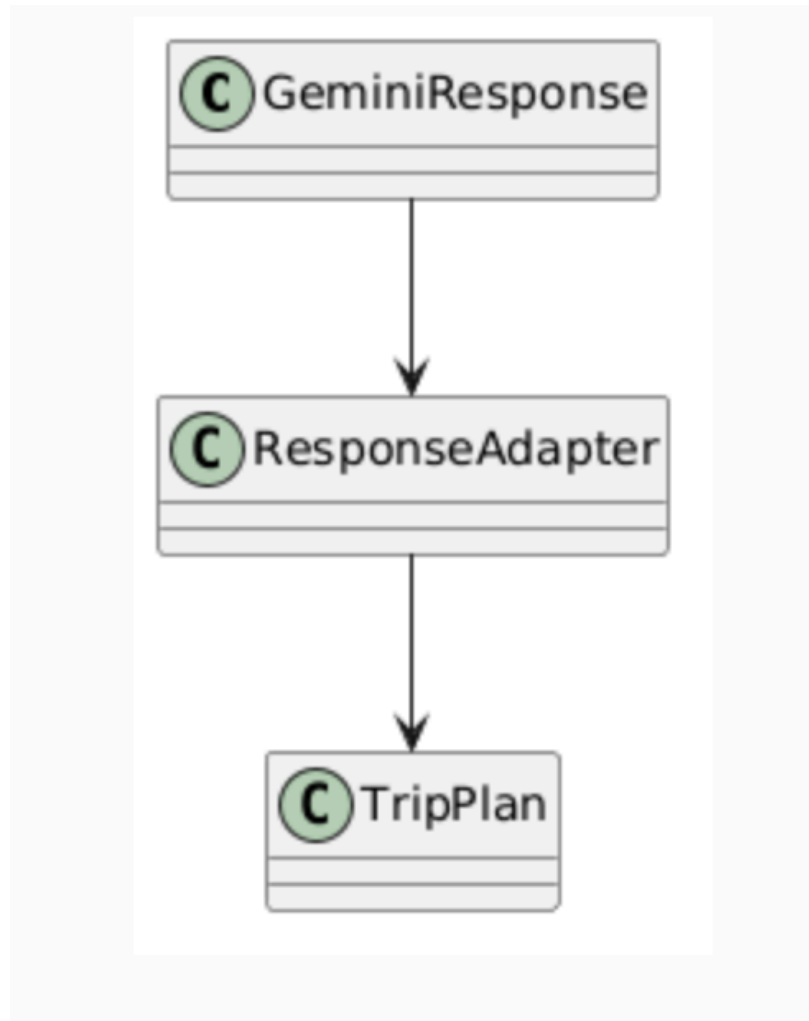
The application receives AI-generated responses that may contain Markdown formatting or non-standard JSON structures.

An adapter-like processing step sanitizes the output and converts it into structured JSON objects that can be stored in the database and used by UI components.

This transformation acts as an **Adapter**, converting external API output into the application's internal data format.

GitHub link:

<https://github.com/zahraaMeky/ai-travel-planner-app/blob/main/app/create-trip/Generate-Trip.jsx#L46-L52>



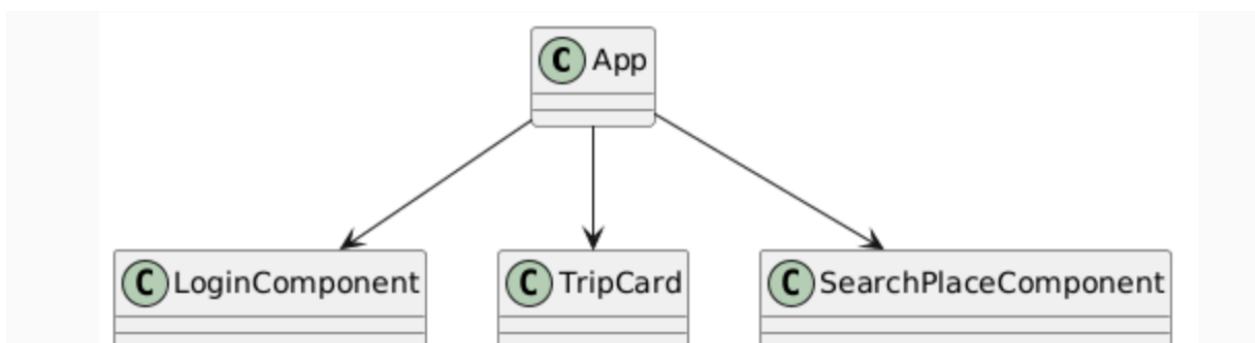
Pattern 5 — Component Pattern (React UI Components)

The application follows the **Component Pattern**, where the UI is built using reusable React components.

Each component encapsulates its own rendering logic and can be reused across multiple screens, improving modularity and maintainability.

GitHub link:

<https://github.com/zahraaMeky/ai-travel-planner-app/blob/main/components/Login.jsx>



b. In your descriptions and/or diagrams, point to the source file and line numbers if applicable, so that the grader can have a look.

5. Design Issues (1 point)

a. Identify one functional design issue and one non-functional design issue.

b. Each design issue requires a descriptive title, the solution option chosen by the project, and the alternative solution that you can think of which will either degrade/improve the design.

Functional Design Issue

- Shipping the Firebase and Gemini keys in the bundled app
 - **Solution Chosen:** The application is set up to bundle both the Firebase DB key and the Gemini AI chatbot key into the app. This is fine if only used by the user compiling but creates a security issue if distributed and have no central authority for the usage of the keys.
 - **Better Solution:** Small backend/proxy for verification of user authorization and communication to services requiring keys.

Non-Functional Design Issue

- Scalability of trips
 - **Solution Chosen:** Query all trips in firebase where “userEmail” == userEmail. This won’t be a big issue right away but if a user starts to accrue many trips eventually the application will begin to slow down over time.
 - **Better Solution:** Only query the top 20 trips connected to the user and if the user scrolls below those already loaded trips, then query for the next 20 trips.

Submission

Please make sure to include team number and the names of all your team members. You are expected to follow the organization of the design documentation with the right information in the right section and appropriate pointers to the project repository content in your descriptions and/or diagrams.

A homework assignment will be created on Gradescope (only one submission per group). You are expected to submit a document in PDF format.

Violation of the above instructions will result in a reduction in score.

